

HyperTrack: Hyperhidrosis Monitoring and Management System

Project Proposal for UCS503

Submitted to: Dr. Jeelani Asif
Thapar Institute of Engineering and Technology
Aditya Rajput, AIML | Prabhkeerat Kaur, AIML

August 2026

1 Higher-order Goal ... secondary framing

This project aims to improve individual health self-management by giving people with hyperhidrosis a structured way to understand their own condition over time. While the underlying motivation is healthcare-adjacent, the immediate engineering objective is to translate unorganized symptom memory into a measurable, deployable software solution: a system that records, analyzes, and reports sweating episodes without making medical claims.

2 Time-to-value ... primary heuristic

- **Ship the core loop first:** episode recording, storage, and a basic history view will be built and validated with users before any analytics or ML feature is attempted.
- **Prioritize fast feedback over completeness:** weekly iterations, with CI-backed testing introduced early, will be used to reduce release friction and catch regressions before they compound.
- **Success is defined incrementally:** what can be delivered and demonstrated at the end of the first iteration (a working record → store → view loop) sets the baseline; dashboards, treatment tracking, reporting, and ML are layered on afterward based on what that first loop reveals.

3 Problem Statement

Hyperhidrosis is a condition characterized by excessive sweating beyond what is needed for normal temperature regulation. It may affect the palms, feet, underarms, or face, and can interfere with academic work, professional tasks, and social interactions.

A major practical difficulty for people experiencing excessive sweating is the lack of a structured way to monitor symptoms over time. Common pain points include:

- **Fragmentation:** individual episodes are remembered in isolation, with no consistent record of severity, duration, affected area, or trigger.

- **Low visibility into trends:** without records, it is difficult to tell whether symptoms are worsening, improving, or tied to specific circumstances (stress, temperature, activity).
- **Nothing to bring to a clinician:** users who do seek medical advice have no organized summary of their history to share, only recollection.
- **No lightweight decision support:** existing general health-tracking apps are not built around hyperhidrosis-specific fields such as affected body area or sweat-specific triggers.

Why this matters now (contextual relevance): even a simple, structured recording tool can meaningfully improve a user's ability to notice patterns and communicate them, without requiring any clinical infrastructure.

4 Proposed Solution ... Software Proposition

4.1 Overview

Build a web-based application, HyperTrack, with the following components:

- **Episode submission:** users record a sweating episode with severity, duration, affected body area, activity, stress level, and optional trigger/notes.
- **Structured storage:** all episodes are stored in a relational database keyed to the user's account.
- **Analytics module:** frequency, average severity, duration, and trigger patterns are computed from stored history.
- **Dashboard:** charts and summaries give the user an at-a-glance view of their trends.
- **Treatment/session tracking:** users can log treatments or sessions they are already following, without the system prescribing anything.
- **Report generation:** a structured, exportable summary of recorded history that can be discussed with a healthcare professional.

4.2 Core Workflow

1. The user logs in and creates a sweating-episode record.
2. The system validates the entered information for completeness and plausibility.
3. The episode is stored in the database.
4. The analytics module processes historical records on demand or on a schedule.
5. The dashboard displays frequency, severity, and trigger trends.
6. The user can generate a structured report at any time.

Data flow: User Input → Episode Recording → Database → Data Analysis → Dashboard → Report

4.3 Operational Constraints for an Educational, Two-person Setting

- **Usability on ordinary devices:** responsive web design so the app works on low-to-mid-range phones and laptops.
- **No unproven research dependencies:** the core system does not depend on novel algorithms; ML is an optional, supplementary layer, not a prerequisite for the core loop.
- **Deployability:** designed to run on common web hosting with a managed database, within the scope of a two-person, one-semester team.

5 Solution Approach ... Engineering Focus

This is a Software Engineering course project, so the emphasis is on requirements analysis, modular architecture, and system correctness rather than novel ML research. The optional ML component is treated as supplementary, not central.

5.1 Severity Analytics ... non-research

Severity and trend analysis will use conventional, explainable methods rather than research-grade modeling:

- **Descriptive statistics:** frequency, average severity, and average duration computed directly from stored records.
- **Trigger frequency:** simple counting/ranking of recorded triggers and affected areas, no claims of causal inference.
- **Optional classification:** a standard, well-understood model (Random Forest as the initial candidate) for severity-category prediction, shown to users as a supplementary insight, never auto-applied as a diagnosis.

5.2 Web Architecture ... web-based solution

A conventional three-tier architecture:

- **Presentation layer:** HTML, CSS, and JavaScript provide the user interface.
- **Application layer:** Python with Flask provides backend services and REST APIs.
- **Data layer:** MySQL stores users, episodes, triggers, treatment records, and analytical results.

This separation keeps the system testable, maintainable, and extendable as features are added iteratively.

5.3 CI/CD and Fast Delivery Enabler

CI/CD will be used to:

- Automatically build and run tests on every change to the backend and analytics modules.
- Produce repeatable deployment artifacts for a staging environment.

- Enable safe, frequent releases of incremental features (episode recording first, dashboard next, ML last).

6 Evaluation Criterion ...measurable and attributable

Success is defined using measurable metrics tied to the core recording-and-insight workflow, rather than subjective impressions alone.

6.1 Primary Evaluation Metric

Episode-logging completion rate (ELCR): the proportion of started episode entries that are completed and successfully stored, measured over a pilot window.

- **Target:** $\geq 90\%$ completion rate for started entries during the pilot.
- **Attribution:** measured from application logs comparing “entry started” events to “entry saved” events in the database.

6.2 Secondary Evaluation Metrics

Metric	Target	Attribution / How measured
Dashboard load time	≤ 2 seconds for a user’s full history	Measured from request timestamp to render-complete event
Report generation accuracy	100% of report fields match underlying stored records	Automated comparison of report output against database records
User-reported clarity	Average $\geq 4/5$ on a simple post-use survey	In-app 1–5 feedback question after report generation
ML severity-prediction accuracy	Reported honestly, not gated on a fixed target	Accuracy, precision, recall, F1-score, confusion matrix on a held-out set
System reliability	$\geq 99\%$ uptime for login and episode submission during testing	Uptime monitoring during pilot window

6.3 Pilot Validation Plan ...high-level

- Recruit 8–15 pilot users (peers or volunteers willing to log sample or real episodes over a short window).
- Compare metrics before/after the first iteration by logging baseline expectations via short interviews before development, then comparing to instrumented data after.

7 Scalability ...with foresight

The proposition should scale in both theoretical and practical senses.

7.1 Scalable (theoretically)

- **Decoupled components:** frontend and backend are separated behind a REST API, so either can evolve independently.

- **Indexed queries:** episode records are indexed by user and date to keep dashboard and report queries fast as history grows.
- **Stateless API design:** backend services do not hold session state in process memory, allowing multiple instances if load increases.

7.2 Operationally deployable (practically)

- **Managed database:** MySQL hosted on a managed service rather than a self-administered server.
- **Platform-hosted or containerized deployment:** suitable for common web hosting within a student budget.
- **CI/CD pipeline:** for staging and, eventually, production releases.

8 Engine Availability Heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project, so the team focuses on domain workflow and correctness rather than inventing infrastructure:

- **Web framework:** Flask for REST APIs and server-rendered/reactive pages.
- **Managed relational database:** MySQL for structured episode, user, and treatment data.
- **Authentication approach:** token- or session-based auth using well-established libraries.
- **Data processing:** Pandas and NumPy for cleaning and aggregating episode records.
- **Machine learning:** Scikit-learn for the optional severity-classification prototype.
- **Visualization:** Chart.js or Matplotlib for dashboard charts.
- **Test framework:** a standard Python testing framework for unit and integration tests.
- **CI runner and staging target:** Git/GitHub-based CI with a staging deployment for pre-release verification.

9 Project Scope and Deliverables ...iteration-friendly

9.1 Initial Deliverable ...first iteration

- User registration/login and episode submission form
- Episode storage with validation
- Basic history view (list of past episodes)
- Logging/timestamps to support the ELCR metric
- CI: build + backend unit tests + linting
- CD to staging with a single command or pipeline job

9.2 Subsequent Deliverables

- Dashboard with frequency, severity, and trigger-pattern charts
- Treatment/session tracking module
- Structured report generation (exportable summary)
- Optional ML severity-prediction prototype with evaluation metrics
- Improved search/filtering and indexed queries for scale readiness

10 Risks and Mitigations

- **Limited data for ML:** initial development may use a public or synthetic dataset; the core recording/analytics loop does not depend on ML being ready.
- **Data privacy:** only necessary fields are collected; data is protected through authentication and standard database security practices.
- **Medical interpretation:** the application will not claim to diagnose or treat hyperhidrosis; all analytics are described as patterns in the user’s own recorded data.
- **ML reliability:** predictions are presented as supplementary insight, evaluated and reported honestly, never as medical advice.
- **Development delays:** iterative delivery prioritizes the core recording loop before dashboard, reporting, and ML features.
- **User adoption / logging fatigue:** the episode-entry form is kept short by default, with optional fields for users who want to add more detail.

11 Summary

HyperTrack is proposed as a software-based monitoring and management system for individuals experiencing hyperhidrosis-related excessive sweating. It addresses the lack of structured symptom monitoring by letting users record episodes and contextual information, then storing, analyzing, visualizing, and summarizing that data into reports.

The project is primarily a Software Engineering system, developed with a time-to-value mindset: a working record–store–view loop first, instrumented with measurable evaluation criteria (episode-logging completion rate, dashboard performance, report accuracy), then extended toward dashboards, treatment tracking, reporting, and an optional AIML severity-prediction component. The architecture is designed to be both theoretically scalable and operationally deployable using a mature, readily available technology stack.

Aditya Rajput Roll No: 1024240145 Email: arajput_be24@thapar.edu
Prabhkeerat Kaur Roll No: 1024240024 Email: pkaur2_be24@thapar.edu